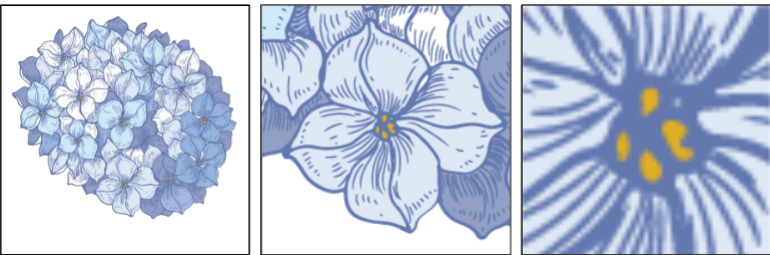


SW 3 - Image File Format

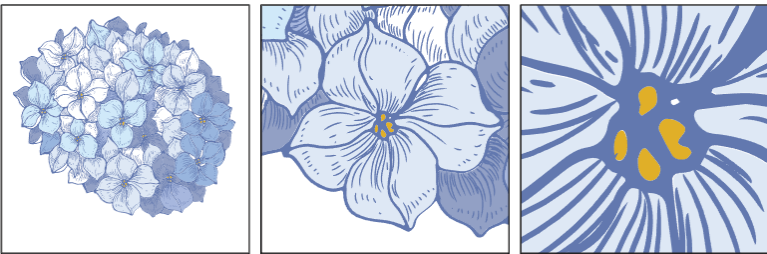
Raster graphic

- The type of image we have dealt with so far
- A grid of pixels, can be represented as matrix of numeric values
- Image sensors naturally capture rasterized image data
- Raster graphics are characterized by a resolution (e.g., pixels per inch: PPI)
- Enable fine-grained color/image processing
- Files are generally larger than vector graphics
- Common file formats include JPEG, PNG, GIF, BMP, and TIFF



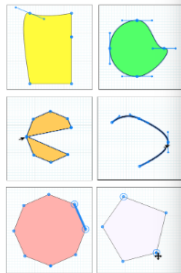
Vector graphic

- Is composed of paths, lines, curves, and shapes based on mathematical equations.
- Content of a vector graphic is stored as a set of instructions that describe how to draw the image.
- Resolution-independent: scale to any size without losing quality
- The display of a vector graphic is more involved; a rendering process is required to convert the mathematical instructions into pixels (that will be shown on the screen)
- Common file formats: SVG, EPS, PDF



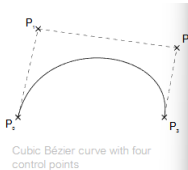
Vector graphic – Components

- **Point:** A coordinate in space
- **Line:** A straight segment connecting two points
- **Curve:** A smooth, bent segment connecting two or more points
 - Examples: Arcs, spline curves, Bézier curves
- **Path:** A sequence of lines and curves
- **Shape** (or polygon): A closed path that encloses a 2D area
- **Text:** A collection of shapes representing characters
- **Style specification:** Visual attributes applied to objects
 - Examples: Fill/stroke color, opacity, gradients, patterns, line styles
- **(Raster image):** Pixel-based content can be included within a vector graphic

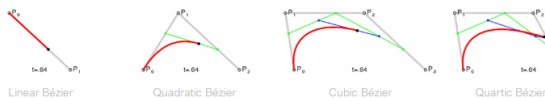


Bézier Curves

- A Bézier curve is a smooth, parametric curve defined by a set of control points.
- Mathematically, Bézier curves are defined as follows:
 - Linear: $B(t) = P_0 + t(P_1 - P_0) = (1-t)P_0 + tP_1, 0 \leq t \leq 1$
 - Quadratic: $B(t) = P_0 + (1-t)^2(P_1 - P_0) + t^2(P_2 - P_0), 0 \leq t \leq 1$
 - Cubic: $B(t) = (1-t)^3P_0 + 3(1-t)^2tP_1 + 3(1-t)t^2P_2 + t^3P_3, 0 \leq t \leq 1$
 - ...

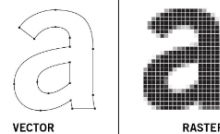


Cubic Bézier curve with four control points



- Foto speichern? → JPEG
- Transparenz nötig? → PNG
- Web, modern? → WebP
- Skalierbar/Logo? → SVG
- Wissenschaftliche Daten? → TIFF / HDF5
- Klinik/Röntgen? → DICOM
- Neuroimaging Forschung? → NIFTI

Vector vs. raster graphics



Hint: Save Matplotlib figures as vector graphics

```
from pathlib import Path
import matplotlib.pyplot as plt

def save_figure(fig=None, path="figure.pdf", **kwargs):
    if fig is None:
        fig = plt.gcf()
    kwargs.setdefault("transparent", True)
    kwargs.setdefault("bbox_inches", "tight")
    kwargs.setdefault("dpi", 300)
    plt.savefig(path, **kwargs)

# Usage:
fig, ax = plt.subplots()
# ...

# Save current figure
save_figure(path="plot.svg")

# Save specific figure
save_figure(fig=fig, path="plot.svg")
```

- Saving figures in vector formats is straightforward with Matplotlib.
- Supported formats: PDF, SVG, EPS
- Pro tip:
 - Problem: Fonts are converted to paths in PDFs by default, making text difficult to edit in vector graphics editors.
 - Solution: To preserve editable text, set the font type to TrueType or Type 42 before exporting the PDF:

```
import matplotlib as mpl
mpl.rcParams["pdf.fonttype"] = 42
```

Raster vs. Vector

	Raster	Vector
Aufbau	Pixelgitter (Matrix)	Mathematische Pfade/Kurven
Skalierung	✗ Qualitätsverlust	✓ verlustfrei
Dateigröße	größer	kleiner
Formate	JPEG, PNG, BMP, TIFF, GIF	SVG, EPS, PDF
Anwendung	Fotos, Mikroskopie	Logos, Diagramme

Raster-Formate im Vergleich

Format	Kompression	Transparenz	Besonderheit
BMP	keine	✗	simpel, grosse Dateien
PNG	lossless (DEFLATE)	✓ Alpha	Standard für Web/Screenshots
JPEG	lossy (DCT, ~10:1)	✗	Fotos, einstellbare Qualität
GIF	lossless	✓ 1-bit	max. 256 Farben, Animationen
TIFF	beides (LZW/JPEG)	✓	Publishing, Medizin, Multi-Page
WebP	beides	✓	25-35% kleiner als JPEG/PNG

JPEG-Kompression (wichtig für Prüfung!)

Bild $\xrightarrow{8 \times 8 \text{ Blöcke}}$ DCT $\xrightarrow{\text{Quantisierung}}$ Entropy coding $\rightarrow$.jpg

DCT = Discrete Cosine Transform (Variante der DFT, nur reelle Werte) → hohe Frequenzen werden weggeworfen → sichtbare Artefakte bei starker Kompression.

Medizinische Formate

Format	Anwendung
DICOM (.dcm)	Klinik: CT, MRI, X-ray — kombiniert Bilddaten + Patientenmetadaten
NIFTI (.nii)	Forschung: 3D/4D Gehirnschans, einfacher als DICOM

Vector-Komponenten (SVG/EPS)

Punkt → Linie → Kurve (Bézier) → Pfad → Shape → Text + Style

Bézier-Kurven parametrisch mit Kontrollpunkten:

- Linear: 2 Punkte
- Quadratisch: 3 Punkte
- Kubisch: 4 Punkte (häufigste in der Praxis)

Compression

- Image file formats support compression to reduce the file size
- Lossless compression:
 - Reduces the file size **without losing image quality**
 - Use when you **prioritize image quality over file size**
- Lossy compression:
 - Reduces **the file size** by discarding some **image data**
 - Goal: discard **"invisible" information** or small details
 - Some methods to gradually set the **compression level**
 - Trade-off: Quality vs. file size

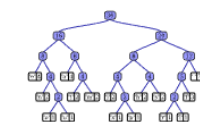
Lossless - Methods

- Run Length Encoding (RLE)
 - Encode consecutive data elements as **a single value and count**
 - Example:
 - Input: WWWWWWWWWWWBWWWWWWWWWWBWWWWWWWWWWB
 - Output: 12W1B12W3B
 - Used in BMP and other bitmap formats
- Huffman-Coding
 - Is a **dictionary-based compression algorithm**: Assign variable-length codes to input data
 - The coding is based on the **frequency of occurrence of data elements**
 - Frequent sequences** are assigned short codes
 - Rarer sequences are assigned longer codes
 - Used in PNG and JPEG files

Example: Huffman coding.

Idea: Use a variable-length code for different tokens (or byte sequences), with the shortest codes used for the most frequent tokens

Input: "this is an example of a huffman tree". Tokens by frequency



Output: Huffman code table

Char	Freq	Code	Char	Freq	Code
space	7	111	s	2	1011
t	4	010	i	2	0110
h	4	100	e	1	11001
l	3	1101	a	1	100110
n	2	1010	p	1	10011
r	2	1000	f	1	11000
m	2	1111	o	1	100111
x	2	1010	l	1	10010

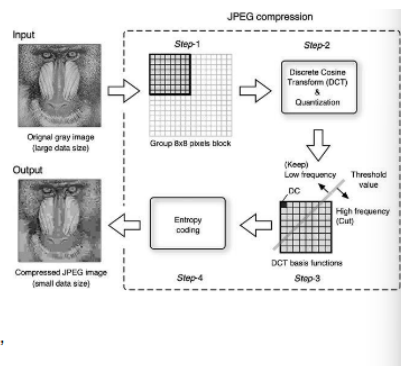
- Lempel-Ziv-Welch (LZW)
 - A **dictionary-based compression algorithm**:
 - Replace **repeated data sequences** with shorter codes
 - Used in GIF, TIFF files

- ZIP
 - Is related to LZW: Replace **repeated data sequences** with shorter codes
 - Makes use of optimize compression algorithms, like **Deflate**
 - Combined LZW and Huffman coding
 - Used in TIFF files

Lossy - Methods

- Divide image into **square blocks of pixels**
- Use the **Discrete Cosine Transform (DCT)** to convert **pixel values into frequency coefficients**
- Quantize **the frequency coefficients** to reduce the amount of data (lossy)
- Use (lossless) Huffman coding to **encode the quantized coefficients**
- Allows for **different levels of compression** by adjusting the quantization tables

- Notes:
 - Other types of transforms (DFT, Wavelets) are possible, but less common



QR: Quick response

Was ist ein QR Code?

2D-Barcode (1994 erfunden), inspiriert vom japanischen Brettspiel Go. Kodiert **numerische**, **alphanumerische** und **binäre Daten** — bis zu **~3'000 Zeichen**, version-abhängig.

Struktur

Komponente	Funktion
Positioning / Orientation	3 Ecken-Quadrate → Erkennung & Ausrichtung
Timing marks	Alternierende schwarz/weiss Streifen → definiert das Pixelgitter
Alignment pattern	Korrigiert Perspektiv-Verzerrungen
Format information	Speichert Error-Correction-Level + Maskmuster
Version information	Version 1-40
Spacing	Ruhezone am Rand → bessere Erkennbarkeit
Datenbits	Der eigentliche codierte Inhalt

Versionen & Grösse

$$\text{Grösse} = (17 + 4 \times \text{Version}) \times (17 + 4 \times \text{Version}) \text{ Module}$$

Version	Grösse	Kapazität
1	21×21	klein
4	33×33	mittel
40	177×177	~3'000 Zeichen

Error Correction (Reed-Solomon)

Level	Name	Wiederherstellbar
L	Low	~7%
M	Medium	~15%
Q	Quartile	~25%
H	High	~30%

→ Deshalb funktionieren QR Codes noch mit Logo in der Mitte oder wenn sie physisch beschädigt sind!

Ein QR Code ist im Grunde ein **selbst-beschreibendes Bild**: er trägt alle Infos mit, die ein Scanner braucht um ihn zu finden, auszurichten, Fehler zu korrigieren und den Inhalt zu lesen — alles ohne externe Datenbank.



- Positioning / Orientation**
Required for the QR code to be detected and localized
- Format information**
Stores data about the error correction level and mask pattern
- Timing marks**
Used to define the pixel grid
- Version information**
QR code version: 1...40
- Spacing**
For better detectability
- Alignment pattern**
Used to correct perspective skew and other distortions in the image.

QR code: Error correction

- Built-in error correction allows reading even if the code is partially damaged
- Core ideas:
 - Add extra, redundant data in the form of mathematical patterns (Reed-Solomon codes)
 - This enables the QR scanner to **detect** and **correct** errors in the data
 - Depending on the correction level, up to ~30% of the code can be recovered